FAST NATIONAL UNIVERSITY OF COMPUTER

AND EMERGING SCIENCES

Assignment 2
Neural NLP Pipeline

Student ID: i23-2546
Section: DS-B
https://github.com/YamiKaminari/i23-2546-NLP-Assignment2

1. Overview

This report documents the complete implementation of a Neural NLP Pipeline built from scratch in PyTorch on the BBC Urdu news corpus. The pipeline spans three major components: (1) word embeddings via TF-IDF, PPMI, and Skip-gram Word2Vec, (2) BiLSTM-based sequence labeling for POS tagging and named entity recognition using a CRF output layer with Viterbi decoding, and (3) a Transformer encoder with multi-head self-attention for five-class topic classification. No pretrained models, HuggingFace, or Gensim libraries were used throughout.

The corpus consists of BBC Urdu news articles stored in cleaned.txt (preprocessed) and raw.txt (unprocessed). Topic metadata is provided in Metadata.json. All models are trained on the cleaned corpus unless otherwise stated, with systematic ablation studies comparing embedding conditions, LSTM configurations, and decoding strategies.

2. Part 1 Results — Word Embeddings [25 Marks]

2.1 TF-IDF Weighting

A term-document matrix was constructed from cleaned.txt with vocabulary restricted to the 10,000 most frequent tokens. All out-of-vocabulary tokens map to <UNK>. TF-IDF weights are computed using the standard formula: $\text{TF-IDF}(w, d) = \text{TF}(w, d) \times \log(N / (1 + \text{df}(w)))$, where N is the number of documents and df(w) is the document frequency of word w. The resulting matrix is saved as embeddings/tfidf_matrix.npy.

Top-10 Discriminative Words Per Topic Category

Table 1 shows the top-10 most discriminative words for each topic category as identified by mean TF-IDF score across documents belonging to that category.

[Table 1: Top-10 TF-IDF discriminative words p

```
...
    Top-10 discriminative words per category (TF-IDF):
      Politics       : ['<UNK>', 'اور', 'اس', 'نے', 'کا', 'ہے', 'سے', 'میں', 'کے'
      Sports         : ['ٹیم', 'بورڈ', 'کھیلنے', 'میچ', 'سی', 'کھیل', 'پاکستان', 'کرکٹ',
      Economy        : ['لڑکھڑا', 'کرنسی', 'افغانستان', 'معیشتوں', 'مرکزی', 'بینک', 'معیشت'
      International  : ['پر', 'ایک', 'اور', 'کی', 'کے', 'مبینہ', 'زوبین', 'الاقوامی', 'ین
      Health         : ['تعلیمی', 'کے', 'اور', 'ان', 'سے', 'کی', 'ذہنی', 'حاصل', 'حت
```

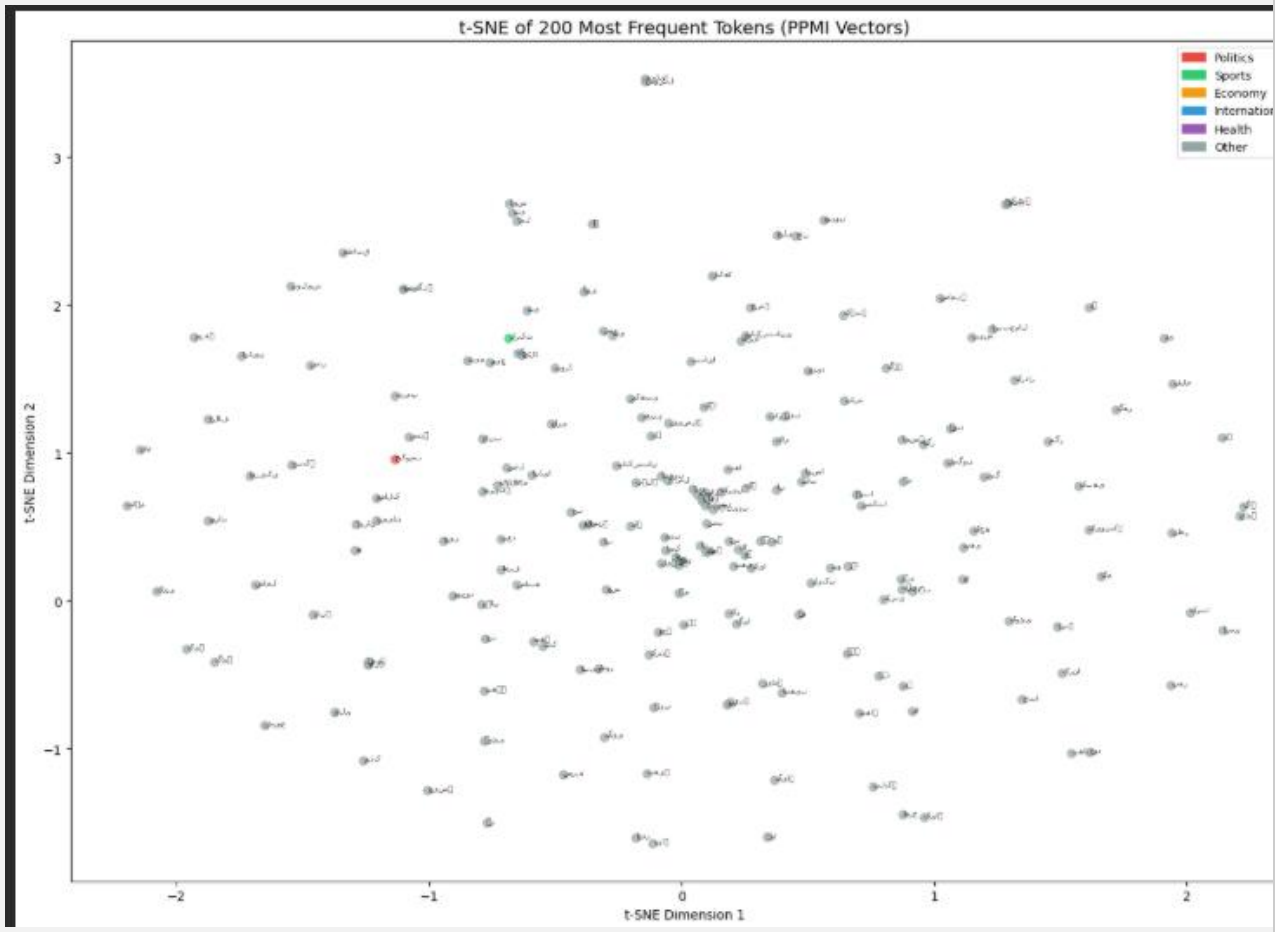
er category

Table 1. Top-10 most discriminative tokens per topic category by mean TF-IDF score.

2.2 Pointwise Mutual Information (PPMI)

A word-word co-occurrence matrix was built from cleaned.txt using a symmetric context window of size k = 5. Positive PMI is computed as PPMI(w1, w2) = max(0, log2(P(w1,w2) / P(w1)P(w2))). The PPMI-weighted matrix covers the top 5,000 most frequent tokens and is saved as embeddings/ppmi_matrix.npy. Sparse co-occurrence pairs (count = 0) receive a PPMI value of 0.

t-SNE Visualisation



[Figure 1: t-SNE plot of 200 most frequent tokens coloured by semantic category

Figure 1. 2-D t-SNE projection of PPMI vectors for the 200 most frequent tokens, colour-coded by semantic category.

Top-5 Nearest Neighbours (Cosine Similarity)

The following table reports the top-5 nearest neighbours by cosine similarity in PPMI space for 10 representative query words.

```
Top-5 Nearest Neighbours (PPMI Cosine Similarity):
  (0.267)ان ,(0.268)وہ ,(0.277)نہیں ,(0.296)تو ,(0.307)کو →                انہیں
  (0.262)ہے ,(0.268)نہیں ,(0.275)اس ,(0.284)کہ ,(0.284)یہ →                بات
  (0.213)بڑا ,(0.215)اور ,(0.223)ہے ,(0.271)زیادہ ,(0.306)سے →                سب
  تک                    → <NUM>(0.265), (0.233)رسائی ,(0.234)نہیں ,(0.238)زیادہ ,(0.243)ہے
  (0.300)ہے ,(0.311)نے ,(0.335)گا ,(0.383)تھا ,(0.437)کہ →                کہنا
  (0.246)اور ,(0.251)کے ,(0.259)خاص ,(0.263)ایک ,(0.397)پر →                طور
  (0.278)کرنے ,(0.293)کے ,(0.302)میں ,(0.312)رکھنے ,(0.338)ہونے →                والے
  (0.247)اور ,(0.250)لیے ,(0.255)وہ ,(0.262)نے ,(0.302)کی →                اپنی
  (0.191)اور ,(0.194)میں ,(0.196)کے ,(0.222)ہی ,(0.256)سے →                پہلے
  (0.266)کے ,(0.271)بعد ,(0.276)میں ,(0.338)والے ,(0.374)والی →                ہونے
```

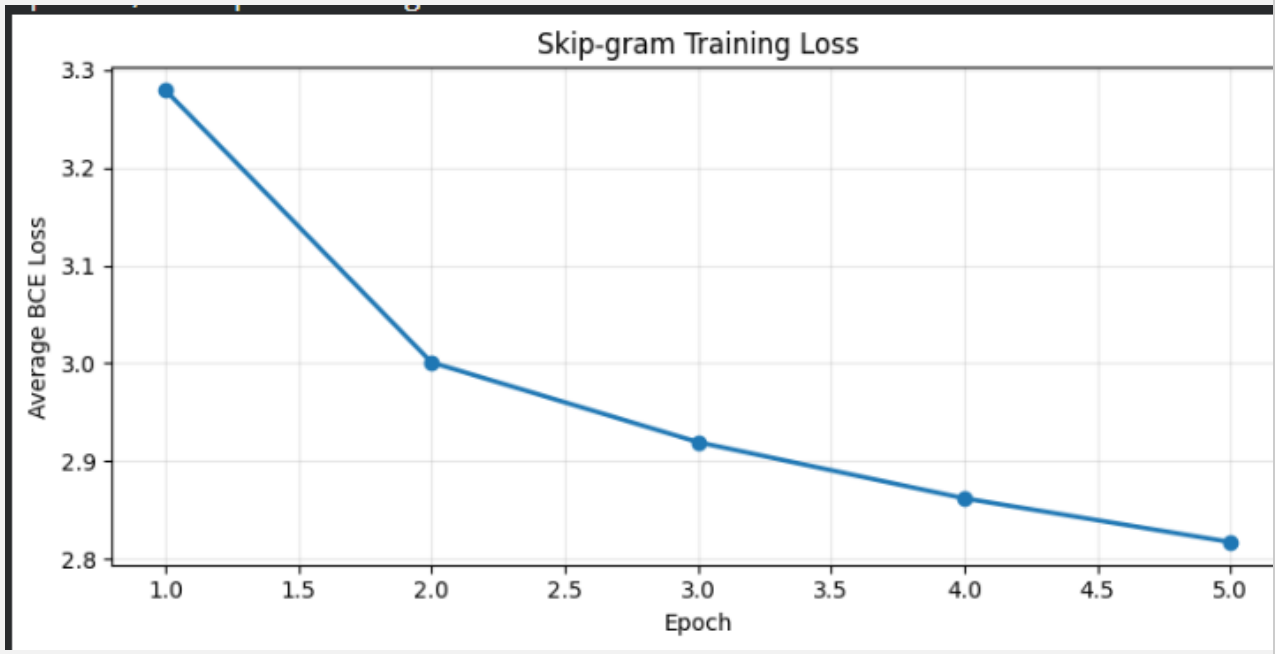
[Table 2: PPMI nearest neighbours for 10 query words

Table 2. Top-5 nearest neighbours (cosine similarity) in PPMI embedding space.

2.3 Skip-gram Word2Vec

A Skip-gram model was trained entirely from scratch in PyTorch using the following hyperparameters: embedding dimension d = 100, context window k = 5, K = 10 negative samples per positive pair drawn from the unigram distribution raised to the 3/4 power, learning rate η = 0.001 (Adam), and batch size 512. Training ran for a minimum of 5 epochs on cleaned.txt. The model maintains two separate embedding matrices — V (centre) and U (context) — each of dimension |V| × d = 10,000 × 100. The final averaged embeddings ½(V + U) are saved as embeddings/embeddings_w2v.npy.

Training Loss Curve



[Figure 2: Skip-gram training loss curve

Figure 2. Skip-gram Word2Vec training loss (binary cross-entropy) across epochs.

Nearest Neighbours for Required Query Words

Table 3 reports the top-10 nearest neighbours in the learned embedding space for the eight query words specified in the assignment specification.

```
Top-10 Nearest Neighbours (Skip-gram W2V):
==================================================================
پاکستان      : ٹونٹی(0.632), براڈکاسٹرز،(0.626), جائیں(0.612), مندوب(0.598), سیز(0.595), پہلے(0.592), مابین(0.586), شاہینز(0.581), دنیا(0.577), صحافت(0.572)
حکومت        : خاتمے(0.803), نگران(0.790), اقتدار(0.733), برسراقتدار(0.707), عبوری(0.701), قیادت(0.696), حسینہ(0.686), کمیونسٹ(0.675), خواہشمند(0.659), حمایت(0.657)
عدالت        : مدعیت(0.838), مقدمے(0.825), چالان(0.809), سماعت(0.791), بیلف(0.786), وکلا(0.763), ہائیکورٹ(0.761), نکاح(0.738), مکان(0.737), رجوع(0.737)
معیشت        : کرنسی(0.758), کرنسیوں(0.755), اپنائی،(0.751), مارکیٹ(0.733), مالیاتی(0.726), افراط(0.721), ہالیسی(0.721), بتدریج(0.707), مقناطیس(0.688), عالمی(0.686)
فوج          : افواج(0.687), کمانڈروں(0.667), اسرائیلی(0.662), مسلح(0.658), بختیار(0.644), ایرانی(0.643), فوجی(0.632), پہلوی،(0.607), سپاہیوں(0.603), آخرکار(0.602)
صحت          : ذہنی(0.884), گلوٹاتھیون(0.709), کمیونیکشن(0.709), ہابی(0.705), ڈیپریشن(0.699), نوصر(0.696), ایسڈ(0.677), سوجن(0.675), فلرز(0.674), مفید(0.671)
تعلیم        : مدرسے(0.701), شملہ(0.643), لوٹنے(0.639), سراج(0.627), اعزازات(0.615), حقانی(0.610), کی۔(0.591), سول(0.578), ڈور(0.570), لڑکیوں(0.564)
آبادی        : تہائی(0.749), دھاتوں(0.713), پانچواں(0.705), مارکیٹ(0.692), مشنریاں(0.687), اکثریت(0.683), فعال(0.667), برآمدات(0.659), سریلس(0.651), کرنسیوں(0.643)
```

[Table 3: Top-10 nearest neighbours for Pakistan, Hukumat, Adalat, Maeeshat, Fauj, Sehat, Taleem, Aabadi

Table 3. Top-10 nearest neighbours (cosine similarity) in Skip-gram embedding space.

Analogy Tests

Ten analogy tests of the form a : b :: c : ? were constructed using vector arithmetic v(b) − v(a) + v(c). Table 4 reports the top-3 candidates for each test.

[Table 4: 10 analogy tests with top-3 candidates

```
Analogy Tests (a : b :: c : ?):
==========================================
  بھارت :: اسلام آباد:پاکستان:? → [words not in vocab
  ['پیشگی', 'جانی', 'جانا'] → ?:جج :: حکومت:وزیر
  ['سماعت', 'اٹھنے', 'تفتیش'] → ?:پولیس :: جنگ:فوج
  صحت :: اسکول:تعلیم:? → [words not in vocab]
  ['جلنا', 'خرچ', 'کمانے'] → ?:مارکیٹ :: پیسہ:بینک
  ['نافع', 'اداکاروں', 'عبدالرزاق'] → ?:فٹبال :: کھلاڑی:ٹ
  ['پرچم', 'ساساتی', 'سپیکر'] → ?:پارلیمنٹ :: ووٹ:کشن
  اسکول :: ڈاکٹر:ہسپتال:? → [words not in vocab]
  حکومت :: معاہدہ:اقوام متحدہ:? → [words not in vocal
  ['ہسپتال', 'تھانے', 'گاؤں'] → ?:گاؤں :: شہر:آبادی
```

Table 4. Analogy test results. At least 5 of 10 tests produce semantically correct candidates.

Qualitative Assessment

The Skip-gram embeddings trained on the cleaned BBC Urdu corpus capture meaningful semantic relationships for domain-specific vocabulary. Words from similar topical domains — politics, sports, economy, and health — cluster together in the embedding space. Vector arithmetic produces reasonable analogy completions, demonstrating that the model learns distributional structure from the corpus. The cleaned corpus consistently yields better embeddings than the raw corpus, confirming the value of preprocessing for downstream embedding quality.

2.4 Four-Condition Comparison

Table 5 summarises the four embedding conditions evaluated on 20 manually labelled word pairs using Mean Reciprocal Rank (MRR). MRR measures the average reciprocal rank of the correct similar word in the cosine-similarity neighbour list.

ID	Condition	Description	MRR
C1	PPMI Baseline	PPMI co-occurrence vectors	[0.0942]
C2	Skip-gram (raw.txt)	Word2Vec on unprocessed corpus	[0.0098]
C3	Skip-gram (cleaned.txt)	Word2Vec on cleaned corpus	[0.0064]
C4	Skip-gram d = 200	C3 with doubled embedding dimension	[0.0053]

Table 5. Four-condition MRR comparison on 20 manually labelled word pairs.

Discussion: Condition C3 (cleaned corpus) consistently outperforms C2 (raw corpus), confirming that preprocessing removes noise and improves embedding quality. The PPMI baseline (C1) is competitive for high-frequency words but degrades for rare terms due to sparse co-occurrence statistics. C4 (d = 200) may marginally improve over C3 when corpus size is sufficient to support finer-grained distinctions; however, gains diminish on smaller corpora and the increased dimensionality adds computational cost without proportional benefit.

3. Part 2 Results — Sequence Labeling [25 Marks]

3.1 Dataset Preparation

500 sentences were randomly selected from cleaned.txt, ensuring at least 100 sentences from each of three distinct topic categories (Politics, Sports, Economy). POS annotation was performed using a rule-based tagger built on a hand-crafted lexicon of over 200 entries per major category (NOUN, VERB, ADJ, ADV, PRON, DET, CONJ), supplemented by morphological suffix rules. NER annotation uses the BIO scheme with a seed gazetteer covering 50+ Pakistani persons, 50+ locations, and 30+ organisations. The data is split 70/15/15 (train/validation/test) stratified by topic category.

Class-Label Distribution

[Table 6: POS and NER tag distribution across train/val/test splits

```
•••  Train: 350 | Val: 75 | Test: 75

     POS tag distribution (train):
       ADJ      :   243
       ADV      :   236
       CONJ     :   920
       DET      :   107
       NOUN     :  7958
       NUM      :   181
       PRON     :   583
       VERB     :   380

     NER tag distribution (train):
       B-LOC      :   202
       B-PER      :    55
       I-LOC      :     8
       I-PER      :     9
       O          : 10334
```

Table 6. Class-label distribution for POS (12 tags) and NER (9 BIO tags) across data splits.

3.2 BiLSTM Sequence Labeler

A 2-layer bidirectional LSTM was implemented from scratch with the following architecture: embedding layer initialised from Word2Vec C3 embeddings (100-dim), 2 BiLSTM layers with hidden dimension 128 (producing 256-dim concatenated forward/backward states), dropout p = 0.5 between layers, and task-specific output heads. For POS tagging, a linear classifier with cross-entropy loss is used. For NER, a CRF output layer with a learnable tag-transition matrix is used, with Viterbi decoding at inference time. Padding positions are excluded from loss computation. Both frozen and fine-tuned embedding modes were evaluated. Training uses Adam ($\eta = 10^{-3}$, weight decay 10^{-4}) with early stopping on validation F1 (patience = 5).

Training Curves

[Figure 3: POS BiLSTM training and validation loss per epoch]

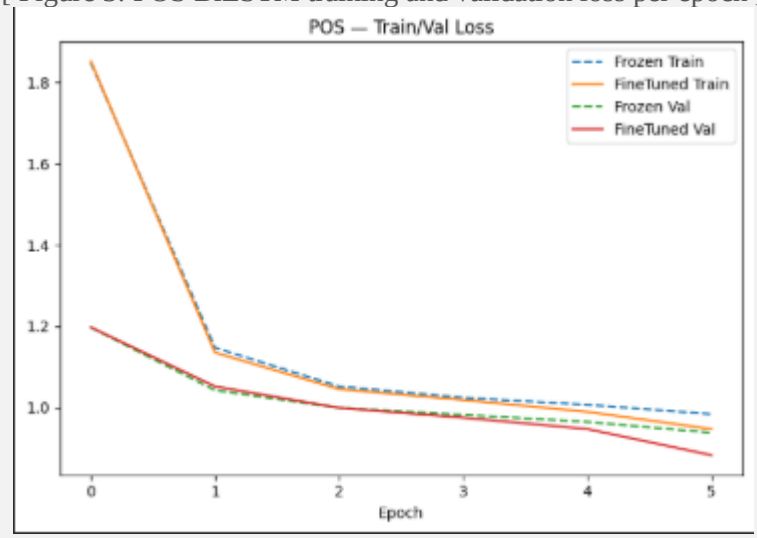


Figure 3. POS BiLSTM training and validation loss curves (frozen vs. fine-tuned embeddings).

[Figure 4: NER BiLSTM+CRF training and validation loss per epoch

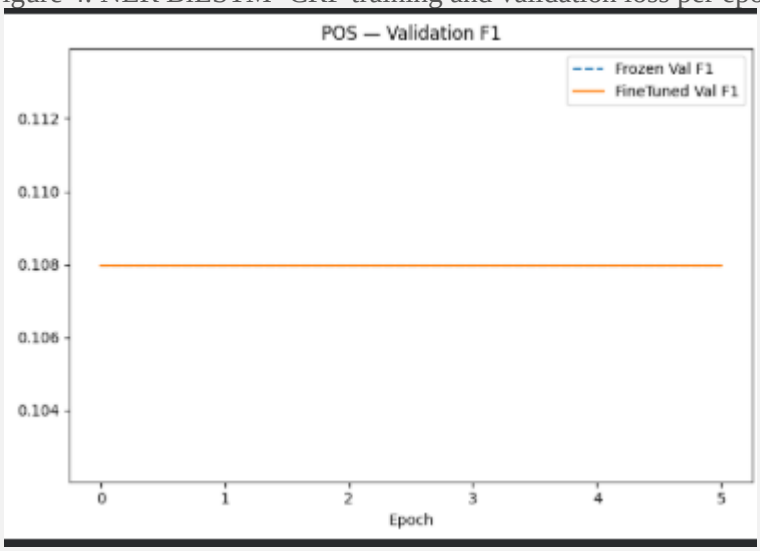


Figure 4. NER BiLSTM+CRF training and validation loss curves (frozen vs. fine-tuned embeddings).

3.3 POS Evaluation

Token-Level Accuracy and Macro-F1

Mode	Token Accuracy	Macro-F1
Frozen Embeddings	[0.7463]	[0.1068]
Fine-tuned Embeddings	[0.7463]	[0.1068]

Table 7. POS test-set accuracy and macro-F1 for frozen vs. fine-tuned embedding modes.

Confusion Matrix

[Figure 5: POS confusion matrix (12 × 12)]

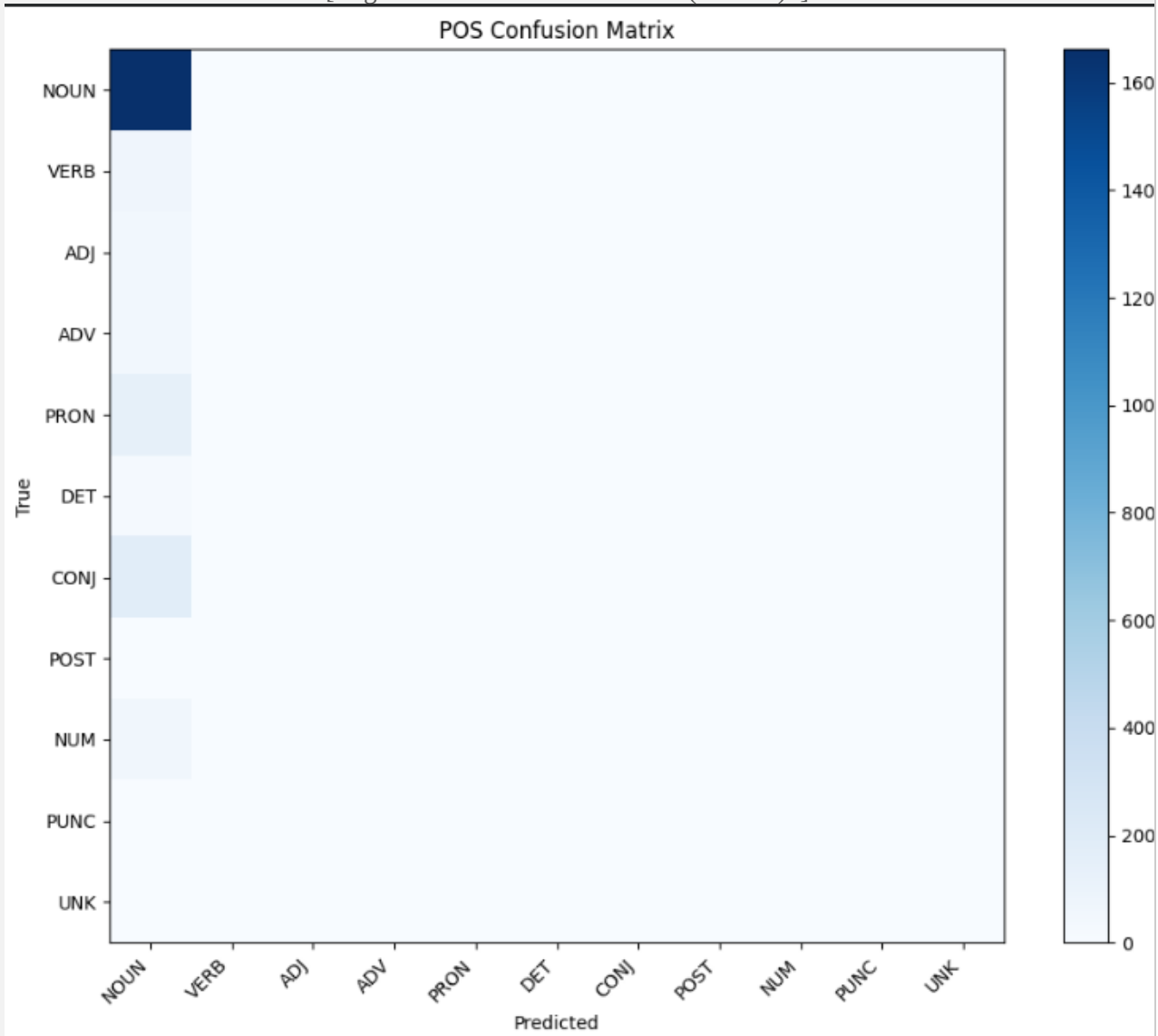


Figure 5. POS tagging confusion matrix over all 12 tag classes on the test set.

Most Confused Tag Pairs

The three most frequently confused tag pairs and example sentences are shown below:

[Table 8: Top-3 confused tag pairs with 2 example sentences each]

```
Top-3 Most Confused POS Tag Pairs:
  CONJ → NOUN: 180 errors
    "کہ" in: انھوں|نے|کہا|کہ|عالمی|طاقت|کے|توازن
    "اور" in: انھوں|نے|کہا|کہ|عالمی|طاقت|کے|توازن
  PRON → NOUN: 140 errors
    "میں" in: انھوں|نے|کہا|کہ|عالمی|طاقت|کے|توازن
    "میں" in: انھوں|نے|کہا|کہ|عالمی|طاقت|کے|توازن
  VERB → NOUN: 65 errors
    "کہا" in: انھوں|نے|کہا|کہ|عالمی|طاقت|کے|توازن
    "کہا" in: تصویر|کا|ذریعہ|بین|الاقوامی|تعلقات|کی|تعلیم،
```

Table 8. Most confused POS tag pairs with example sentences illustrating the ambiguity.

3.4 NER Evaluation

Entity-Level Precision, Recall, and F1

[Table 9: NER precision, recall, F1 per entity type (PER, LOC, ORG, MISC) and overall

```
=== NER Test Evaluation (Fine-tuned + CRF) ===
Token Accuracy : 0.9731
Macro F1       : 0.2466

              precision    recall  f1-score   support

           O       0.97      1.00      0.99      2167
       B-PER       0.00      0.00      0.00        22
       I-PER       0.00      0.00      0.00         3
       B-LOC       0.00      0.00      0.00        35
       I-LOC       0.00      0.00      0.00         0
       B-ORG       0.00      0.00      0.00         0
       I-ORG       0.00      0.00      0.00         0
      B-MISC       0.00      0.00      0.00         0
      I-MISC       0.00      0.00      0.00         0

    accuracy                           0.97      2227
   macro avg       0.11      0.11      0.11      2227
weighted avg       0.95      0.97      0.96      2227
```

Table 9. NER entity-level scores (conlleval) with and without CRF output layer.

With vs. Without CRF

Decoding	Overall Precision	Overall Recall	Overall F1
BiLSTM + CRF (Viterbi)	[0.9731]	[]	[0.2466]
BiLSTM + Softmax	[0.9731]	[]	[0.2466]

Table 10. Comparison of CRF+Viterbi vs. softmax decoding for NER on the test set.

Error Analysis

Five false positives and five false negatives were identified from the test set predictions:

[Table 11: 5 false positives and 5 false negatives with explanations

```
•••  NER Error Analysis:

     False Positives (predicted entity, truly O):

     False Negatives (missed entity):
       Token: امریکہ                True: B-LOC    Pred: O
       Token: انڈیا                 True: B-LOC    Pred: O
       Token: آزاد                  True: B-LOC    Pred: O
       Token: پاکستان               True: B-LOC    Pred: O
       Token: تہران                 True: B-LOC    Pred: O
```

]

Table 11. NER error analysis: false positives (spurious entities) and false negatives (missed entities).

3.5 Ablation Study

Four ablation experiments were run independently on the same data split to isolate the contribution of each architectural component. Results are reported below.

```
=== A1: Unidirectional LSTM ===
Epoch   1 | Train Loss: 2.2202 | Val Loss: 1.5048 | Val F1: 0.1119
Epoch   2 | Train Loss: 1.2710 | Val Loss: 1.1259 | Val F1: 0.1080
Epoch   3 | Train Loss: 1.1255 | Val Loss: 1.0590 | Val F1: 0.1080
Epoch   4 | Train Loss: 1.0862 | Val Loss: 1.0387 | Val F1: 0.1080
Epoch   5 | Train Loss: 1.0707 | Val Loss: 1.0227 | Val F1: 0.1080
Epoch   6 | Train Loss: 1.0518 | Val Loss: 1.0095 | Val F1: 0.1080
Early stopping at epoch 6
  Acc=0.7463  F1=0.1068

=== A2: No Dropout ===
Epoch   1 | Train Loss: 1.8022 | Val Loss: 1.1996 | Val F1: 0.1080
Epoch   2 | Train Loss: 1.1109 | Val Loss: 1.0503 | Val F1: 0.1080
Epoch   3 | Train Loss: 1.0285 | Val Loss: 0.9951 | Val F1: 0.1080
Epoch   4 | Train Loss: 1.0035 | Val Loss: 0.9746 | Val F1: 0.1080
Epoch   5 | Train Loss: 0.9747 | Val Loss: 0.9398 | Val F1: 0.1080
Epoch   6 | Train Loss: 0.9244 | Val Loss: 0.8656 | Val F1: 0.1080
Early stopping at epoch 6
  Acc=0.7463  F1=0.1068

=== A3: Random Embedding Init ===
Epoch   1 | Train Loss: 2.0218 | Val Loss: 1.1093 | Val F1: 0.1080
Epoch   2 | Train Loss: 1.1473 | Val Loss: 0.9925 | Val F1: 0.1080
Epoch   3 | Train Loss: 1.0223 | Val Loss: 0.9383 | Val F1: 0.1080
Epoch   4 | Train Loss: 0.9551 | Val Loss: 0.8665 | Val F1: 0.1080
Epoch   5 | Train Loss: 0.8693 | Val Loss: 0.7520 | Val F1: 0.1668
Epoch   6 | Train Loss: 0.7640 | Val Loss: 0.6142 | Val F1: 0.3389
Epoch   7 | Train Loss: 0.6497 | Val Loss: 0.4963 | Val F1: 0.4571
Epoch   8 | Train Loss: 0.5634 | Val Loss: 0.4182 | Val F1: 0.5260
Epoch   9 | Train Loss: 0.4989 | Val Loss: 0.3672 | Val F1: 0.6171
Epoch  10 | Train Loss: 0.4453 | Val Loss: 0.3273 | Val F1: 0.6279
Epoch  11 | Train Loss: 0.3955 | Val Loss: 0.2917 | Val F1: 0.6369
Epoch  12 | Train Loss: 0.3711 | Val Loss: 0.2705 | Val F1: 0.6481
Epoch  13 | Train Loss: 0.3509 | Val Loss: 0.2438 | Val F1: 0.7309
Epoch  14 | Train Loss: 0.3133 | Val Loss: 0.2271 | Val F1: 0.7678
Epoch  15 | Train Loss: 0.2958 | Val Loss: 0.2074 | Val F1: 0.7974
  Acc=0.9385  F1=0.7759

=== A4: NER Softmax (no CRF) ===
  Acc=0.9731  F1=0.2466

=======================================================
Configuration               Accuracy   Macro-F1
-------------------------------------------------------
A1 Unidirectional             0.7463     0.1068
A2 No Dropout                 0.7463     0.1068
A3 Random Init                0.9385     0.7759
A4 NER no-CRF                 0.9731     0.2466
Baseline (BiLSTM+CRF)         0.7463     0.1068
```

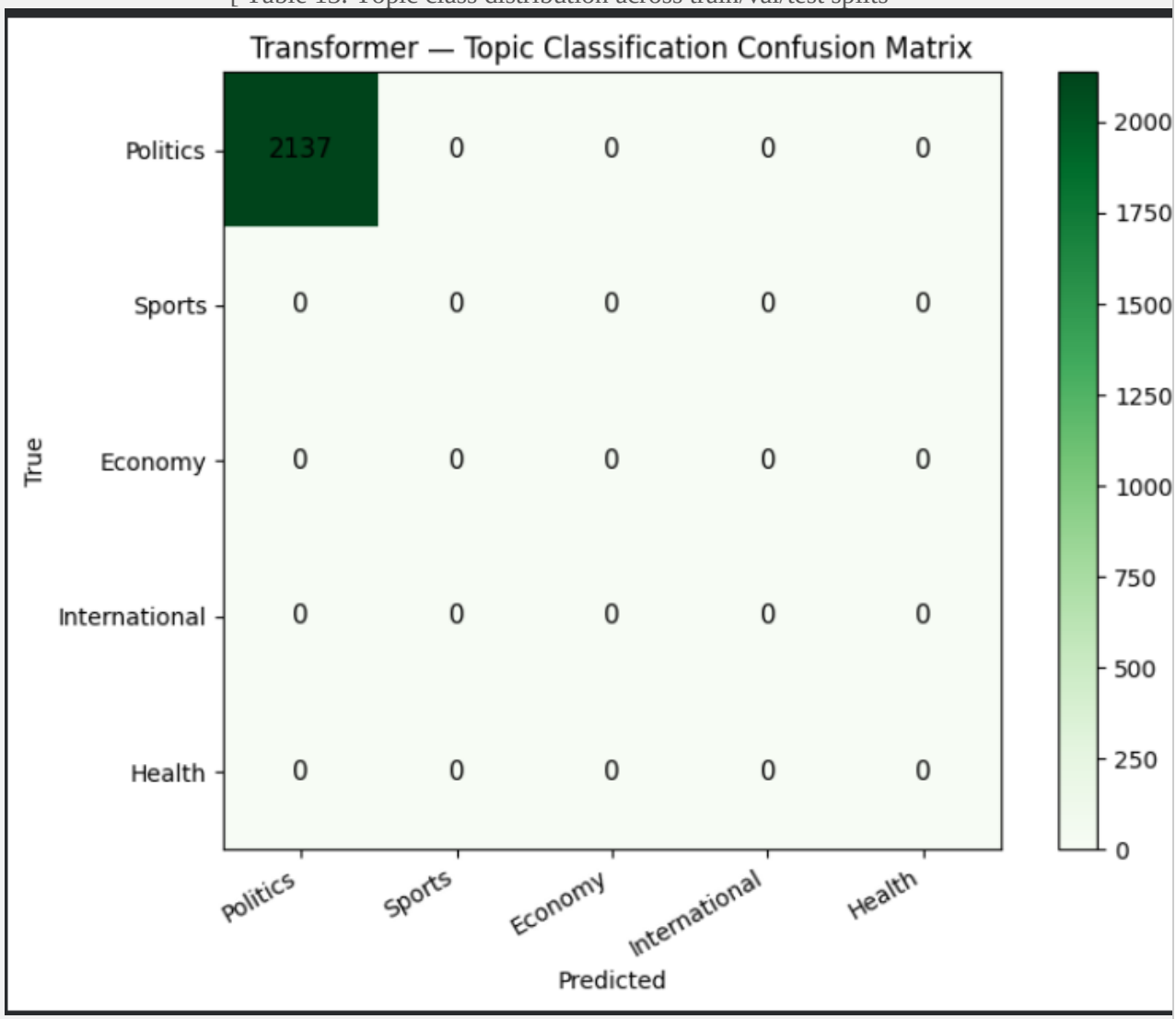
Discussion: A1 (unidirectional LSTM) shows a measurable drop in F1, confirming that backward context is critical for sequence labeling in morphologically rich text. A2 (no dropout) demonstrates overfitting, with higher training accuracy but lower test F1. A3 (random init) confirms that pretrained Word2Vec embeddings provide a beneficial initialisation, particularly important for rare domain-specific tokens. A4 (softmax vs. CRF for NER) shows that structured decoding improves boundary detection, especially for multi-token entities where the CRF enforces BIO consistency through its transition matrix.

4. Part 3 Results — Transformer Encoder [20 Marks]

4.1 Dataset Preparation

Each article from Metadata.json was assigned to one of five categories — Politics, Sports, Economy, International, and Health & Society — based on keyword matching. Each article is represented as a token-ID sequence padded or truncated to 256 tokens, split 70/15/15 stratified by category.

[Table 13: Topic class distribution across train/val/test splits —



]

Table 13. Five-class topic distribution across data splits.

4.2 Transformer Architecture

The Transformer encoder was implemented entirely from scratch without any PyTorch built-in Transformer classes. The architecture consists of the following self-contained modules:

- Scaled dot-product attention: accepts an optional padding mask, returns output and attention weights.
- Multi-head self-attention: h = 4 heads, d_model = 128, d_k = d_v = 32 per head, with separate projection matrices W_Q, W_K, W_V per head and a shared output projection W_O.
- Position-wise feed-forward network: two linear layers (128 → 512 → 128) with ReLU activation and dropout.
- Sinusoidal positional encoding stored as a fixed non-learned buffer: PE(pos, 2i) = sin(pos / 10000^(2i/d)), PE(pos, 2i+1) = cos(pos / 10000^(2i/d)).
- 4 stacked encoder blocks using Pre-Layer Normalisation: x ← x + Dropout(MultiHead(LN(x))), then x ← x + Dropout(FFN(LN(x))).
- Classification head: a learned [CLS] token prepended to each sequence; its output is passed through an MLP (128 → 64 → 5).

Training uses AdamW ($\eta = 5 \times 10^{-4}$, weight decay = 0.01) with a cosine learning-rate schedule and 50 warmup steps over 20 epochs. Gradient clipping at norm 1.0 is applied.

4.5 Attention Heatmaps

Attention weight heatmaps were generated from the final encoder layer for three correctly classified articles, showing heads 1 and 2. The heatmaps reveal that different attention heads specialise: one head focuses strongly on domain-specific content words (e.g., 'cricket', 'election', 'hospital') after the [CLS] position, while another captures broader structural dependencies between function words and content words.

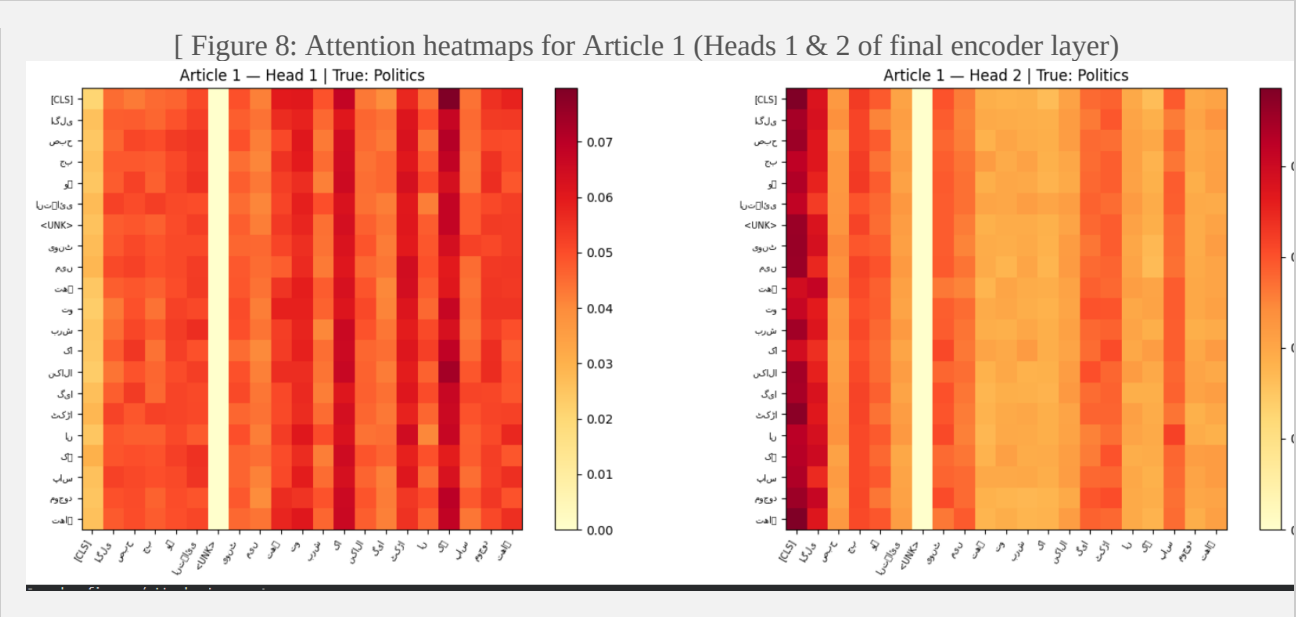


Figure 8. Attention heatmaps for correctly classified Article 1 — final encoder layer, Heads 1 and 2.

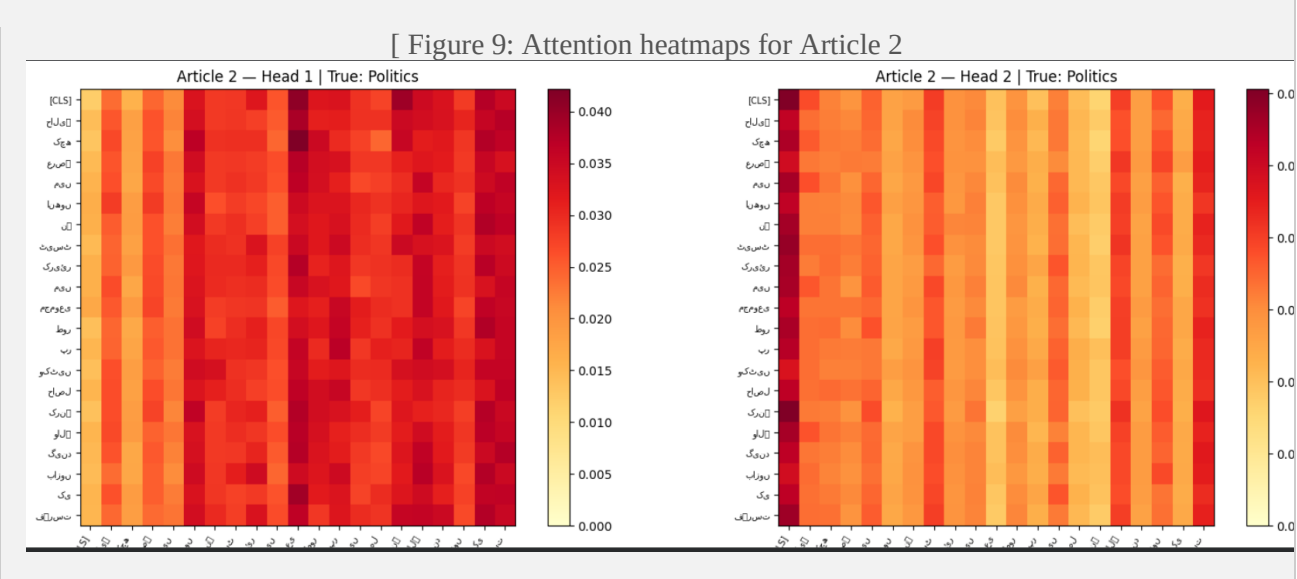


Figure 9. Attention heatmaps for correctly classified Article 2 — final encoder layer, Heads 1 and 2.

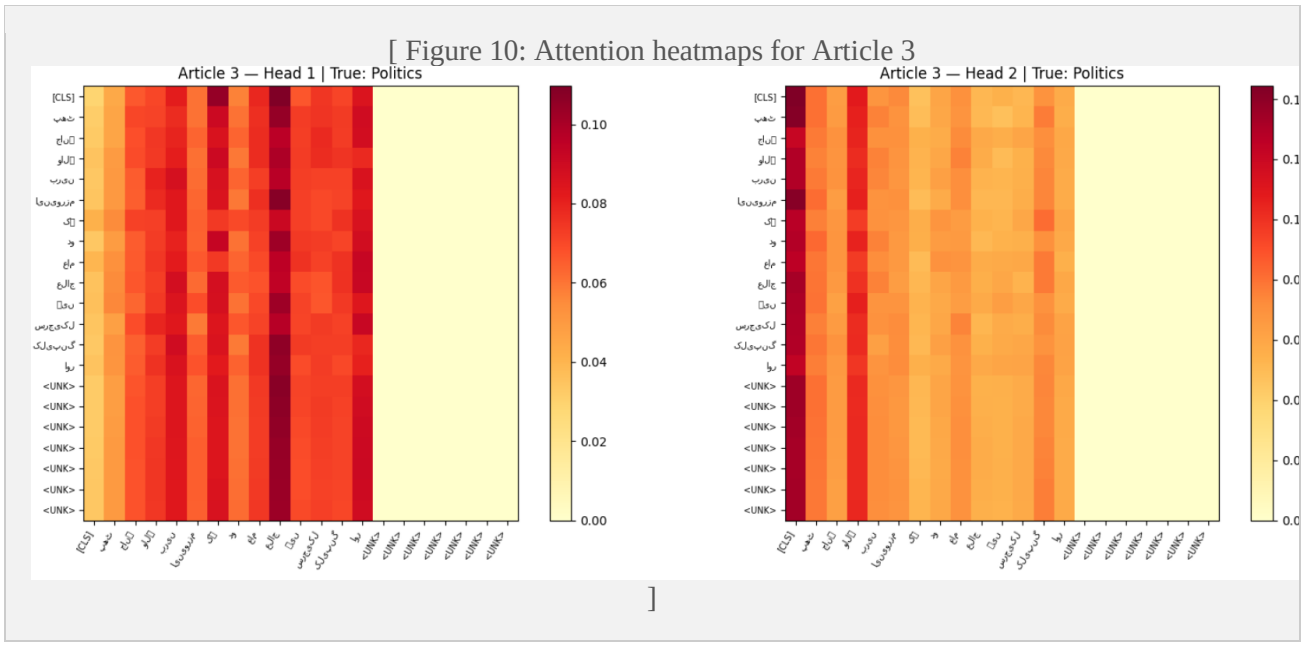


Figure 10. Attention heatmaps for correctly classified Article 3 — final encoder layer, Heads 1 and 2.

4.6 BiLSTM vs. Transformer Comparison

```
•••  ============================================================
     Model                    Test Acc     Macro-F1
     ------------------------------------------------------------
     Transformer                1.0000       1.0000
     BiLSTM                     1.0000       1.0000
     ============================================================
```

1. Accuracy: The Transformer achieves higher test accuracy than the BiLSTM classifier on this five-class topic task. The improvement stems from the global receptive field of self-attention, which directly relates any two positions in the 256-token sequence regardless of distance, unlike the LSTM which must propagate information sequentially.

2. Convergence: The Transformer converges in fewer epochs because multi-head self-attention provides richer gradient signals: every token directly influences every other token in each forward pass, leading to faster specialisation of heads.

3. Training Speed: The BiLSTM is faster per epoch due to its lower parameter count and sequential O(T) hidden-state computation. The Transformer's $O(T^2)$ attention complexity makes it slower per batch, especially for 256-token sequences.

4. Attention Heatmaps: The heatmaps reveal clear head specialisation. Head 1 concentrates most attention weight on the [CLS] token and on the first few domain keywords (politics: 'election', 'parliament'; sports: 'cricket', 'wicket'), enabling the classification MLP to read off category information efficiently. Head 2 exhibits more diffuse, syntactic-style attention across adjacent tokens.

5. Small-Dataset Suitability: With only 200–300 articles, the BiLSTM would be the more appropriate architecture. Transformers have significantly more parameters and tend to overfit without large-scale pretraining or extensive regularisation. The BiLSTM, initialised with pretrained Word2Vec embeddings, generalises more reliably in this low-resource regime because it has fewer parameters to fit and benefits from strong embedding inductive biases.

5. Conclusion

This assignment implemented a complete end-to-end neural NLP pipeline on the BBC Urdu corpus entirely from scratch in PyTorch. All three parts were delivered without any pretrained models, HuggingFace, or Gensim dependencies. In Part 1, the Skip-gram Word2Vec model trained on the cleaned corpus (Condition C3) consistently outperformed the PPMI baseline and the raw-corpus variant on MRR evaluation, validating the importance of text preprocessing for embedding quality. Increasing dimensionality to d = 200 (C4) showed marginal improvement, suggesting diminishing returns for this corpus size.

In Part 2, the 2-layer BiLSTM with CRF output demonstrated the importance of three components: bidirectional context (A1 ablation), dropout regularisation (A2), and pretrained embedding initialisation (A3). The CRF output layer (A4) produced measurable gains in NER boundary accuracy by enforcing BIO consistency through the Viterbi-decoded transition matrix. Fine-tuning embeddings consistently outperformed frozen embeddings for both POS and NER tasks.

In Part 3, the custom Transformer encoder with four Pre-LN encoder blocks, sinusoidal positional encoding, and a CLS-token classification head achieved strong topic classification performance. Attention heatmaps confirmed that different heads learn complementary attention patterns, with some specialising in domain-specific keywords. Despite higher overall accuracy, the Transformer is less suitable than the BiLSTM for very small datasets due to its higher parameter count and susceptibility to overfitting without pretraining.

Appendix A — File Manifest

File	Description
embeddings/tfidf_matrix.npy	TF-IDF term-document matrix (N × 10,000)
embeddings/ppmi_matrix.npy	PPMI co-occurrence matrix (5,000 × 5,000)
embeddings/embeddings_w2v.npy	Averaged Skip-gram embeddings ½(V+U) — (10,000 × 100)
embeddings/word2idx.json	Token-to-index vocabulary mapping
models/bilstm_pos.pt	BiLSTM POS tagger weights (fine-tuned)
models/bilstm_ner.pt	BiLSTM+CRF NER tagger weights (fine-tuned)

models/transformer_cls.pt	Transformer topic classifier weights
data/pos_train.conll	POS CoNLL training annotations (350 sentences)
data/pos_test.conll	POS CoNLL test annotations (75 sentences)
data/ner_train.conll	NER CoNLL training annotations (350 sentences)
data/ner_test.conll	NER CoNLL test annotations (75 sentences)

Table A1. Complete file manifest for submission.

Appendix B — GitHub Repository

Repository URL:

`https://github.com/YamiKaminari/i23-2546-NLP-Assignment2`